
Modulul 2. Reprezentarea datelor

Cuprins

Introducere	25
Obiectivele modului.....	25
U1. Reprezentarea numerelor întregi	26
U2. Operații aritmetice cu numere întregi în cod complementar.....	32
U3. Reprezentarea numerelor raționale și a caracterelor	41



Introducere

Folosirea calculatorului implică și utilizarea unor modalități de reprezentare a informațiilor în memoria lui ca șiruri de biți. Orice informatician trebuie să cunoască cum sunt reprezentate informațiile numerice și alfa-numerice în memorie. De asemenea, este uneori necesar să poată interpreta conținutul locațiilor de memorie. Asupra fiecărui tip de date sunt permise anumite operații. Vom prezenta modul în care se efectuează adunările, scăderile și înmulțirile numerelor întregi reprezentate în cod complementar.



Competențe

La sfârșitul acestui modul studenții vor fi capabili să:

- Determine reprezentarea în memoria calculatorului a numerelor întregi, raționale și a caracterelor;
- Determine ce informație este memorată într-o locație de memorie, cunoscând tipul informației (număr întreg, rațional sau caracter);
- Să efectueze operații aritmetice cu numere întregi reprezentate în cod complementar.

Unitatea de învățare M2.U1. Reprezentarea numerelor întregi

Cuprins

M2.U1.1. Introducere.....	26
M2.U1.2. Obiectivele unității de învățare.....	26
M2.U1.3. Codul direct	27
M2.U1.4. Codul invers.....	28
M2.U1.5. Codul complementar	29
M2.U1.6. Rezumat.....	31
M2.U1.7. Test de evaluare a cunoștințelor	31



M2.U1.1. Introducere

Se știe că în memoria calculatoarelor informațiile sunt reprezentate ca succesiuni de 0 și 1. Deseori informațiile sunt numere întregi, a căror reprezentare va fi studiată în această unitate de învățare. Cele mai cunoscute 3 modalități de reprezentare a numerelor întregi sunt:

1. Codul direct
2. Codul invers
3. Codul complementar

Codul direct este cel mai „apropiat” de modul în care suntem obișnuiți să scriem numerele, însă cel mai eficient este codul complementar, care este folosit pentru reprezentarea numerelor întregi în memoria calculatoarelor.



M2.U1.2. Obiectivele unității de învățare

Această unitate de învățare își propune ca obiectiv principal însușirea de către studenți a celor mai cunoscute 3 modalități de reprezentare a numerelor întregi.

La sfârșitul acestei unități de învățare studenții vor fi capabili să:

- Reprezinte numere întregi în cod direct;
- Reprezinte numere întregi în cod invers;
- Reprezinte numere întregi în cod complementar.



Durata medie de parcurgere a primei unități de învățare este de 2 ore.

M1.U1.3. Codul direct

În reprezentarea în *cod direct* pe n biți a unui număr întreg, cel mai semnificativ bit este rezervat pentru semn, iar ceilalți $n-1$ biți conțin reprezentarea în baza 2 a modulului numărului. Pentru numerele pozitive bitul de semn este 0, iar pentru cele negative bitul de semn este 1.



Exemple

1. Reprezentarea numărului 12 în cod direct pe 8 biți este

$$\begin{array}{c} 0 \quad 0001100 \\ \text{Semn} \quad \text{Mărime} \end{array}$$

2. Reprezentarea numărului -12 în cod direct pe 8 biți este

$$\begin{array}{c} 1 \quad 0001100 \\ \text{Semn} \quad \text{Mărime} \end{array}$$

Se observă cu ușurință că singura deosebire între codul direct al lui 12 și cel al lui -12 este bitul de semn.

Observații 1) Reprezentarea în cod direct a unui număr negativ se obține din reprezentarea modulului numărului prin schimbarea bitului de semn din 0 în 1.

2) Un dezavantaj al codului direct este faptul că +0 and -0 au două reprezentări diferite, deși sunt unul și același număr: 0. De exemplu, codul direct pe 8 biți al lui +0 este 00000000, iar al lui -0 este 10000000.

În consecință, pe n biți pot fi reprezentate în cod direct numai $2^n - 1$ numere întregi diferite (de la $-2^{n-1} + 1$ la $2^{n-1} - 1$) chiar dacă există 2^n șiruri diferite de n biți. De exemplu, pe 8 biți se pot reprezenta folosind codul direct 255 de numere întregi: de la -127 la 127, iar pe 16 biți se pot reprezenta 65535 de numere întregi: de la -32767 la 32767.



Determinați codurile directe pe 8 biți ale următoarelor numere: -127, -56, -3, 3, 5, 111, 127.

Până acum am văzut cum determinăm codul direct al unui număr întreg dat. Acum vom studia problemă inversă: Presupunem că cunoaștem un cod și vrem să aflăm numărul al cărui cod direct se dă.

1. Determinăm numărul binar obținut din codul direct prin eliminarea bitului de semn, îl convertim în baza 10 și obținem modulul numărului căutat. Fie acesta m .
2. Dacă bitul de semn este 0 atunci numărul căutat este chiar m , iar dacă bitul de semn este 1 atunci numărul este $-m$.



Exemple

- Determinăm numărul întreg al cărui cod direct este 00010101.
Deoarece bitul de semn este 0, înseamnă că este un număr pozitiv, care este echivalentul zecimal al numărului binar 0010101. Acesta este 21.
- Determinăm numărul întreg al cărui cod direct este 10010101.
Deoarece bitul de semn este 1, înseamnă că este un număr negativ, al cărui modul este echivalentul zecimal al numărului binar 0010101, adică 21. Deci, numărul căutat este -21.

M2.U1.4. Codul invers

Ca și în reprezentarea în cod direct, în reprezentarea în *cod invers* pe n biți a unui număr întreg, cel mai semnificativ bit este rezervat pentru semn. Pentru numerele pozitive bitul de semn este 0, iar pentru cele negative bitul de semn este 1. Pentru numerele pozitive, ceilalți $n-1$ biți conțin reprezentarea lor în binar. Codul invers al unui număr negativ se obține din codul invers al modulului său prin înlocuirea biților 0 cu 1 și a biților 1 cu 0. Această operație se numește *complementarea biților*.



Exemple

1. Reprezentarea numărului 12 în cod invers pe 8 biți este 00001100.
2. Reprezentarea numărului -12 în cod invers pe 8 biți este 11110011.

Observații 1) Pentru numere întregi pozitive, reprezentarea în cod invers coincide cu cea în cod direct.

2) Un dezavantaj al reprezentării în cod invers este, ca și în cazul codului direct, faptul că +0 and -0 au două reprezentări diferite, deși sunt unul și același număr: 0. De exemplu, codul invers pe 8 biți al lui +0 este 00000000, iar al lui -0 este 11111111.

În consecință, pe n biți pot fi reprezentate în cod invers, ca și în cod direct, numai 2^n-1 numere întregi diferite (de la $-2^{n-1}+1$ la $2^{n-1}-1$) chiar dacă există 2^n șiruri diferite de n biți. De exemplu, pe 8 biți se pot reprezenta folosind codul invers 255 de numere întregi: de la -127 la 127, iar pe 16 biți se pot reprezenta 65535 de numere întregi: de la -32767 la 32767.

Reprezentarea în cod invers nu este folosită în mod obișnuit decât ca pas intermediar în determinarea codului complementar.



Determinați codurile inverse pe 8 biți ale următoarelor numere: -127, -56, -3, 3, 5, 111, 127.

Până acum ne-am pus problema determinării codului invers al unui număr întreg dat. Acum vom studia problemă inversă: Presupunem că știm un cod și vrem să aflăm numărul al cărui cod invers se cunoaște.

1. Dacă este numărul reprezentat este pozitiv (adică dacă bitul de semn este 0), atunci convertim codul invers în baza 10 și obținem numărul.
2. Dacă este numărul reprezentat este negativ (adică dacă bitul de semn este 1), atunci aplicăm aceeași metodă pe care am aplicat-o pentru determinarea codului invers: înlocuim toți biții 0 cu 1 și toți biții 1 cu 0. Am obținut acum codul unui număr pozitiv, care este modulul numărului negativ pe care-l căutăm. Acesta se obține prin convertirea codului în baza 10.



Exemple

- Determinăm numărul întreg al cărui cod invers este 00010101.
Deoarece bitul de semn este 0, înseamnă că este un număr pozitiv, care este echivalentul zecimal al numărului binar 0010101. Acesta este 21.
- Determinăm numărul întreg al cărui cod invers este 10010101.
Deoarece bitul de semn este 1, înseamnă că este un număr negativ, al cărui modul este echivalentul zecimal al numărului binar 01101010 obținut prin inversarea biților. Acesta este 104. Deci, numărul căutat este -104.

M2.U1.5. Codul complementar

Ca și în codurile direct și invers, în reprezentarea în *cod complementar* pe n biți a unui număr întreg, cel mai semnificativ bit este rezervat pentru semn. Pentru numerele pozitive bitul de semn este 0, iar pentru cele negative bitul de semn este 1. Pentru numerele pozitive, ceilalți $n-1$ biți conțin reprezentarea lor în binar. Codul complementar al unui număr negativ se obține din său codul invers la care se adună 1. Ultimii n biți ai acestei sume reprezintă codul complementar al numărului negativ dat.

Majoritatea calculatoarelor lucrează cu numere reprezentate în cod complementar.



Exemple

1. Reprezentarea numărului 12 în cod complementar pe 8 biți este 00001100.
2. Reprezentarea numărului -12 în cod complementar pe 8 biți este

$$\begin{array}{r} 11110011+ \quad // \text{codul invers al lui } -12 \\ \underline{1} \\ 11110100 \quad // \text{codul complementar al lui } -12. \end{array}$$

Observații 1) Pentru numere întregi pozitive, reprezentările în cod direct, invers și complementar coincid.

2) În cod complementar +0 și -0 au aceeași reprezentare. De exemplu, codul complementar pe 8 biți al lui +0 este 00000000, iar al lui -0 este format din ultimii 8 biți ai sumei

$$\begin{array}{r} 11111111+ \\ \underline{1} \\ 100000000 \end{array}$$

Adică, codul complementar al lui -0 este 00000000 ca și cel al lui $+0$.

În consecință, pe n biți pot fi reprezentate în cod complementar 2^n numere întregi diferite: de la -2^{n-1} la $2^{n-1}-1$. De exemplu, pe 8 biți se pot reprezenta folosind codul complementar 256 de numere întregi: de la -128 la 127 , iar pe 16 biți se pot reprezenta 65536 de numere întregi: de la -32768 la 32767 . Adică, datorită unicei reprezentări pentru $+0$ și -0 , se câștigă încă un număr negativ, -2^{n-1} , care poate fi reprezentat în cod complementar pe n biți, dar care nu putea fi reprezentat nici în cod direct, nici în cod invers pe n biți.

O altă metodă de determinare a codului complementar al unui număr negativ este următoarea:

1. Se determină codul complementar al modulului numărului.
2. Pornind de la dreapta spre stânga:
 - Toți biții 0 rămân neschimbați până la întâlnirea primului 1
 - Primul bit 1 rămâne neschimbat
 - Toți ceilalți biți vor fi schimbați: biții 1 vor deveni 0, iar biții 0 vor deveni 1.

Până acum ne-am pus problema determinării codului complementar al unui număr întreg dat. Acum vom studia problemă inversă: Presupunem că se cunosc cei n biți prin care este memorat un anumit număr întreg și vrem să aflăm numărul.

1. Dacă este numărul reprezentat este pozitiv (adică dacă bitul de semn este 0), atunci convertim codul complementar în baza 10 și obținem numărul.
2. Dacă este numărul reprezentat este negativ (adică dacă bitul de semn este 1), atunci aplicăm aceeași metodă pe care am aplicat-o pentru determinarea codului complementar: mai întâi înlocuim toți biții 0 cu 1 și toți biții 1 cu 0, apoi adunăm 1 și păstrăm ultimii n biți ai sumei. Aceștia trebuie să conțină reprezentarea în cod complementar a unui număr pozitiv, care se obține prin convertirea codului în baza 10 și care reprezintă modulul numărului negativ pe care-l căutăm.



Exemple

- Determinăm numărul întreg al cărui cod complementar este 00010101.
Deoarece bitul de semn este 0, înseamnă că este un număr pozitiv, care este echivalentul zecimal al numărului binar 00010101. Acesta este 21.
- Determinăm numărul întreg al cărui cod complementar este 10010101.
Deoarece bitul de semn este 1, înseamnă că este un număr negativ. Inversăm biții și obținem 01101010. Apoi adunăm 1 și obținem 01101011. Acesta este codul complementar al unui număr pozitiv, care este modulul numărului pe care îl căutăm. Deoarece $01101011_{(2)} = 107_{(10)}$, rezultă că numărul căutat este -107 .



Determinați codurile complementare pe 8 biți ale următoarelor numere:
-127, -56, -3, 3, 5, 111, 127.



M2.U1.6. Rezumat

Cele mai cunoscute modalități de reprezentare a numerelor întregi sunt: codul direct, codul invers și codul complementar.

Majoritatea calculatoarelor lucrează cu numere reprezentate în cod complementar.

În toate cele 3 coduri cel mai semnificativ bit este folosit pentru semn: 0 pentru numere pozitive și 1 pentru cele negative.

Pentru numere întregi pozitive, reprezentările în cod direct, invers și complementar coincid și sunt egale cu reprezentarea numerelor în binar.

Codul complementar al unui număr negativ se obține din său codul invers la care se adună 1. Ultimii n biți ai acestei sume reprezintă codul complementar al numărului negativ dat.



M2.U1.7. Test de evaluare a cunoștințelor

1. Determinați codurile directe pe 8 biți ale următoarelor numere:
11, 12, +0, -0, -111.
2. Determinați codurile inverse pe 8 biți ale următoarelor numere:
11, 12, +0, -0, -111
3. Determinați codurile complementare pe 8 biți ale următoarelor numere:
11, 12, +0, -0, -111.

Unitatea de învățare M2.U2. Operații aritmetice cu numere întregi în cod complementar

Cuprins

M2.U2.1. Introducere.....	32
M2.U2.2. Obiectivele unității de învățare.....	32
M2.U2.3. Adunarea numerelor în cod complementar	32
M2.U2.4. Înmulțirea numerelor în cod complementar	34
M2.U2.5. Rezumat.....	39
M2.U2.6. Test de evaluare a cunoștințelor	40



M2.U2.1. Introducere

Majoritatea calculatoarelor lucrează cu numere întregi reprezentate în cod complementar. Vom studia adunarea și înmulțirea întregilor în cod complementar. Scăderea poate fi realizată prin adunarea descăzutului cu opusul scăzătorului.



M2.U2.2. Obiectivele unității de învățare

Această unitate de învățare își propune ca obiectiv principal însușirea de către studenți a modului în care calculatorul efectuează operații aritmetice cu numere întregi reprezentate în cod complementar.

La sfârșitul acestei unități de învățare studenții vor fi capabili să:

- Să efectueze adunări, scăderi și înmulțiri de numere întregi în cod complementar.



Durata medie de parcurgere a acestei unități de învățare este de 3 ore.

M2.U2.3. Adunarea numerelor întregi în cod complementar

Majoritatea calculatoarelor utilizează numerele întregi reprezentate în cod complementar pentru că acest cod are multiple avantaje. Unul dintre ele a fost deja menționat în unitatea de învățare anterioară: unica reprezentare a lui 0. Alt avantaj este faptul că cele mai rapide circuite pot fi folosite dacă numerele sunt reprezentate în cod complementar. Circuitul pentru scădere poate fi eliminat, deoarece scăderea poate fi efectuată cu ajutorul circuitului destinat adunării, cu o mică modificare.

Circuitul destinat adunării, numit și *sumator*, adună două numere în cod complementar. Transportul către bitul de semn și transportul de la bitul de semn sunt

comparate. Dacă aceste ultime două transporturi sunt egale atunci suma (fără transportul de la bitul de semn) este corectă. În caz contrar, rezultatul este incorect și programatorul va primi un mesaj de eroare (depășire).



Exemplu

Vom arăta cum se adună 2 numere pozitive reprezentate în cod complementar pe 8 biți.

$$\begin{array}{r} \\ [22]_c = 00010110+ \\ [34]_c = \underline{00100010} \\ 00111000 \end{array}$$

Transportul către bitul de semn și transportul de la bitul de semn sunt ambele 0, deci rezultatul corect. Deoarece bitul de semn al sumei este 0, înseamnă că s-a obținut un număr pozitiv. Acesta este 56, deoarece $00111000_{(2)} = 56_{(10)}$.



Exemplu

Vom arăta cum se adună 2 numere negative reprezentate în cod complementar pe 8 biți.

$$\begin{array}{r} \\ [-22]_c = 11101010+ \\ [-34]_c = \underline{11011110} \\ 111001000 \end{array}$$

Transportul către bitul de semn și transportul de la bitul de semn sunt ambele 1, deci rezultatul, care constă în ultimii 8 biți ai sumei, este corect. Deoarece bitul de semn al sumei este 1, înseamnă că s-a obținut un număr negativ. Valoarea modulului acestui număr negativ este 56, deoarece $00111000_{(2)} = 56_{(10)}$. Această ultimă transformare a fost făcută doar din motive didactice. Calculatoarele nu o mai efectuează deoarece numerele negative sunt memorate în cod complementar.



Exemplu

Vom arăta cum se adună un număr pozitiv cu unul negativ în cod complementar pe 8 biți, suma fiind pozitivă.

$$\begin{array}{r} \\ [-22]_c = 11101010+ \\ [34]_c = \underline{00100010} \\ 100001100 \end{array}$$

Transportul către bitul de semn și transportul de la bitul de semn sunt ambele 1, deci rezultatul, care constă în ultimii 8 biți ai sumei, este corect. Deoarece bitul de semn al sumei este 0, înseamnă că s-a obținut un număr pozitiv. Valoarea acestui număr este 12, deoarece $00001100_{(2)} = 12_{(10)}$.



Exemplu

Vom arăta cum se adună un număr pozitiv cu unul negativ în cod complementar pe 8 biți, suma fiind un număr negativ.

$$\begin{array}{r} [22]_c = 00010110+ \\ [-34]_c = \underline{11011110} \\ \hline 11110100 \end{array}$$

Transportul către bitul de semn și transportul de la bitul de semn sunt ambele 0, deci rezultatul, care constă în ultimii 8 biți ai sumei, este corect. Deoarece bitul de semn al sumei este 1, înseamnă că s-a obținut un număr negativ. Valoarea acestui număr negativ este -12, deoarece $00001100_{(2)} = 12_{(10)}$. Această ultimă transformare a fost făcută doar din motive didactice. Calculatoarele nu o mai efectuează deoarece numerele negative sunt memorate în cod complementar.



Exemplu

Vom arăta cum apare o eroare de depășire. Vom încerca să adunăm 73 cu 65 în cod complementar pe 8 biți.

$$\begin{array}{r} [73]_c = 01001001+ \\ [65]_c = \underline{01000001} \\ \hline 10001010 \end{array}$$

Transportul către bitul de semn este 1, iar transportul de la bitul de semn este 0, ceea ce înseamnă că a avut loc o depășire și, ca urmare, rezultatul este incorect. Într-adevăr, bitul de semn al sumei este 1, ceea ce înseamnă că s-a obținut un număr negativ, -108, care, evident nu poate fi rezultatul unei adunări corecte a două numere pozitive. Eroarea a apărut din cauză că suma, care era 138, nu este un număr care poate fi reprezentat pe 8 biți în cod complementar.

Pentru scădere majoritatea calculatoarelor nu au un circuit special, deoarece scăderea poate fi efectuată cu ajutorul circuitului destinat adunării, cu o mică modificare.



Exemplu

Scăderea $22 - 33$ poate fi transformată în adunarea $22 + (-33)$.

$$\begin{array}{r} [22]_c = 00010110+ \\ [-33]_c = \underline{11011111} \\ \hline 11110101 \end{array}$$

Transportul către bitul de semn și transportul de la bitul de semn sunt ambele 0, deci rezultatul, care constă în ultimii 8 biți, este corect. Deoarece bitul de semn al sumei este 1, înseamnă că s-a obținut un număr

negativ. Valoarea acestui număr negativ este -11 , deoarece $00001011_{(2)} = 11_{(10)}$. Această ultimă transformare a fost făcută doar din motive didactice. Calculatoarele nu o mai efectuează deoarece numerele negative sunt memorate în cod complementar.



1. Efectuați următoarele adunări în cod complementar pe 8 biți:

$$56+12$$

$$-127+51,$$

$$-3+3,$$

$$5+(-111).$$

M2.U2.4. Înmulțirea numerelor întregi în cod complementar

Unele calculatoare efectuează înmulțirile prin adunări repetate. Dar, majoritatea calculatoarelor au circuite speciale pentru înmulțire. În general, se implementează diferite variante ale algoritmului clasic de înmulțire.

Algoritmul de înmulțire

Acest algoritm este asemănător cu algoritmul pe care îl folosim pentru înmulțirea a 2 numere întregi utilizând creionul și hârtia: pentru fiecare cifră a înmulțitorului, se scrie o sumă parțială cu o poziție la stânga, sub precedenta sumă parțială. Înmulțirea cu 1 înseamnă copierea de înmulțitului cu o poziție la stânga, sub precedenta sumă parțială. Înmulțirea cu 0 înseamnă scrierea un șir de zerouri cu o poziție la stânga, sub precedenta sumă parțială. În final, rezultatul se obține adunând toate sumele parțiale.

Algoritmul folosește doi regiștri dubli. Un *registru* este un dispozitiv de stocare temporară a informației, având, de obicei 16 biți și fiind accesat de către unitatea aritmetico-logică. Unul dintre regiștrii dubli este un registru de deplasare. Un *registru de deplasare* poate deplasa fiecare bit memorat cu o poziție spre stânga, completând cel mai puțin semnificativ bit cu 0 și eliminând cel mai semnificativ bit.

Algoritmul de înmulțire a două numere întregi pozitive reprezentate în cod complementar pe n biți este următorul:

Algoritm inmultire;

begin

se pune de înmulțitul în registrul de deplasare SR;

se inițializează cu 0 registrul dublu DR care va conține rezultatul;

if cel mai puțin semnificativ bit al înmulțitorului este 1 **then**

DR = DR + SR;

for fiecare din cei $n-1$ biți rămași ai înmulțitorului, mergând de la dreapta spre stânga
do

begin

se deplasează de înmulțitul din SR cu o poziție spre stânga;

if bitul curent al înmulțitorului este 1 **then**

```

DR = DR + SR;
end;
if cei mai semnificativi  $n+1$  biți ai registrului DR sunt 0 then
    cei mai puțin semnificativi  $n$  biți ai registrului DR conțin produsul corect;
else produsul nu poate fi reprezentat pe  $n$  biți;
end.

```



Exemplu

Vom aplica algoritmul pentru a înmulți 13 cu 6 în cod complementar pe 8 biți.

```

[13]c = 00001101
[6]c = 00000110
Inițial,
SR: 00000000 00001101
DR: 00000000 00000000

```

Cel mai puțin semnificativ bit al înmulțitorului este 0, deci la primul pas nu facem nimic.

Pentru bitul subliniat al înmulțitorului 00000110:

Se deplasează de înmulțitul din registrul SR cu o poziție spre stânga:

SR: 00000000 00011010

Se adună conținutul celor 2 regiștri, rezultatul păstrându-se în registrul

DR:

DR: 00000000 00011010

Pentru bitul subliniat al înmulțitorului 00000110:

Se deplasează de înmulțitul din registrul SR cu o poziție spre stânga:

SR: 00000000 00110100

Se adună conținutul celor 2 regiștri, rezultatul păstrându-se în registrul

DR:

DR: 00000000 01001110

Deoarece ceilalți biți ai înmulțitorului sunt 0, conținutul registrul DR nu se va mai modifica. Deci, nu mai are rost să efectuăm ultimii 5 pași ai algoritmului, care constau doar în deplasări ale de înmulțitului din registrul SR.

Deoarece cei mai semnificativi 9 biți ai registrului DR sunt 0, înseamnă că cei mai puțin semnificativi 8 biți ai lui DR conțin rezultatul corect, care este 78.

Observație Dacă unul sau ambii operanzi sunt negativi, atunci el sau ei pot fi înlocuiți cu modulul lor înainte de începerea algoritmului, iar la sfârșitul algoritmului rezultatul se înlocuiește cu opusul său dacă este necesar.



Exemplu

Putem determina rezultatul înmulțirii lui -11 cu 9 în cod complementar pe 8 biți aplicând algoritmul de înmulțire pentru 11 și 9 și determinând apoi opusul produsului furnizat de algoritm.

$$[11]_c = 00001011$$

$$[9]_c = 00001001$$

Inițial,

SR: 00000000 00001011

DR: 00000000 00000000

Cel mai puțin semnificativ bit al înmulțitorului este 1, deci vom aduna SR la DR.

DR: 00000000 00001011

Pentru bitul subliniat al înmulțitorului 00001001:

Se deplasează deînmulțitul din registrul SR cu o poziție spre stânga:

SR: 00000000 00010110

Pentru bitul subliniat al înmulțitorului 00001001:

Se deplasează deînmulțitul din registrul SR cu o poziție spre stânga:

SR: 00000000 00101100

Pentru bitul subliniat al înmulțitorului 00001001:

Se deplasează deînmulțitul din registrul SR cu o poziție spre stânga:

SR: 00000000 01011000

Se adună conținutul celor 2 regiștri, rezultatul păstrându-se în registrul

DR:

DR: 00000000 01100011

Deoarece ceilalți biți ai înmulțitorului sunt 0, conținutul registrul DR nu se va mai modifica. Deci, nu mai are rost să efectuăm ultimii 4 pași ai algoritmului, care constau doar în deplasări ale deînmulțitului din registrul SR.

Deoarece cei mai semnificativi 9 biți ai registrului DR sunt 0, înseamnă că cei mai puțin semnificativi 8 biți ai lui DR, 01100011, conțin rezultatul corect al înmulțirii lui 11 cu 9. Pentru a obține produsul dintre -11 și 9 vom determina opusul numărul furnizat de algoritm, care este 10011101, adică reprezentarea în cod complementar pe 8 biți a lui -99.



Efectuați următoarele înmulțiri în cod complementar pe 8 biți:

$$5 * 12$$

$$(-17) * 4,$$

$$(-3) * (-19),$$

$$5 * (-11).$$

Algoritmul lui Booth

Se știe că

$$2^m + 2^{m-1} + \dots + 2^{n+1} + 2^n = 2^{m+1} - 2^n,$$

pentru orice $n, m \in \mathbb{N}$, $m > n$.

În consecință, pentru un șir de biți 1 consecutivi ai înmulțitorului, în loc de un șir de adunări, se pot efectua doar o singură adunare și o singură scădere, obținându-se același rezultat.

Algoritmul lui Booth folosește 3 regiștri: M , A și Q . Inițial, M conține de înmulțitul, toți biții registrului A sunt 0, iar Q conține înmulțitorul.

Folosind metoda lui Booth, se poate implementa un circuit pentru înmulțire, care utilizează cei mai puțin semnificativi 2 biți ai registrului Q . Dacă acești 2 biți sunt:

0	0	atunci se efectuează o deplasare a lui AQ
0	1	atunci se adună M la A și se efectuează o deplasare a lui AQ
1	0	atunci se scade M din A și se efectuează o deplasare a lui AQ
1	1	atunci se efectuează o deplasare a lui AQ

În acest algoritm, o deplasare a lui AQ înseamnă deplasarea fiecărui bit cu o poziție la dreapta, pierderea celui mai puțin semnificativ bit și păstrarea celui mai semnificativ bit.



Exemplu

Aplicăm algoritmul lui Booth pentru a înmulți 15 cu 3 în cod complementar pe 8 biți.

$[15]_c = 00001111$

$[3]_c = 00000011$

M	A	Q	Q_{prec}	Operații
00001111	00000000 11110001	00000011	0	Scădere și deplasare
	11111000	10000001	1	deplasare
	11111100 00001011	01000000	1	adunare și deplasare
	00000101	10100000	0	deplasare
	00000010	11010000	0	deplasare
	00000001	01101000	0	deplasare
	00000000	10110100	0	deplasare
	00000000	01011010	0	deplasare
	00000000	00101101	0	

Toți biții din registrul A sunt zero și cel mai semnificativ bit al lui Q este tot 0, deci Q conține codul complementar al unui număr pozitiv, 45, care este produsul corect.

Algoritmul lui Booth este mai eficient decât algoritmul de înmulțire dacă unul dintre operanzi, care va fi înmulțitorul, conține unul sau câteva șiruri de biți 1 consecutivi.

Observație Algoritmul lui Booth se poate folosi și dacă unul sau ambii operanzi sunt numere întregi negative.



Exemplu

Aplicăm algoritmul lui Booth pentru a înmulți 41 cu -2 în cod complementar pe 8 biți.

$$[41]_c = 00101001$$

$$[-2]_c = 11111110$$

M	A	Q	Q_{prec}	Operații
00101001	00000000	1111111 <u>0</u>	<u>0</u>	deplasare
	00000000 11010111	0111111 <u>1</u>	<u>0</u>	scădere și deplasare
	11101011	1011111 <u>1</u>	<u>1</u>	deplasare
	11110101	1101111 <u>1</u>	<u>1</u>	deplasare
	11111010	1110111 <u>1</u>	<u>1</u>	deplasare
	11111101	0111011 <u>1</u>	<u>1</u>	deplasare
	11111110	1011101 <u>1</u>	<u>1</u>	deplasare
	11111111	0101110 <u>1</u>	<u>1</u>	deplasare
	11111111	1010111 <u>0</u>	<u>1</u>	

Interpretarea rezultatului: toți biții din registrul A sunt 1 și cel mai semnificativ bit al lui Q este tot 1, deci Q conține codul complementar al unui număr negativ, reprezentabil pe 8 biți. Modul acestui număr este echivalentul zecimal al numărului binar 01010010, care este 82.



Efectuați următoarele înmulțiri în cod complementar pe 8 biți, folosind algoritmul lui Booth:

$$5 * 15$$

$$(-17) * 6,$$

$$(-3) * (31),$$

$$25 * (-2).$$



M2.U2.5. Rezumat

Majoritatea calculatoarelor utilizează numerele întregi reprezentate în cod complementar datorită multiplelor avantaje pe care le are acest cod: 0 are o reprezentare unică, iar cele mai rapide circuite pot fi folosite dacă numerele sunt

reprezentate în cod complementar.

Circuitul destinat adunării, numit și *sumator*, adună două numere în cod complementar. Transportul către bitul de semn și transportul de la bitul de semn sunt comparate. Dacă aceste ultime două transporturi sunt egale atunci suma (fără transportul de la bitul de semn) este corectă. În caz contrar, rezultatul este incorect și programatorul va primi un mesaj de eroare (depășire).

Circuitul pentru scădere poate fi eliminat, deoarece scăderea poate fi efectuată cu ajutorul circuitului destinat adunării, cu o mică modificare.

Înmulțirea numerelor întregi reprezentate în cod complementar se poate face folosind un algoritm de înmulțire asemănător celui pe care îl folosim când înmulțim numerele scriind cu creionul pe hârtie sau folosind algoritmul lui Booth. Acesta din urmă este eficient a fi aplicat atunci când unul dintre operanzi, care va fi înmulțitorul, conține unul sau câteva șiruri de biți 1 consecutivi.



M2.U2.6. Test de evaluare a cunoștințelor

1. Adunați numerele:

19 și 62,

45 și -65,

-111 și -56

în cod complementar pe 8 biți.

2. Scădeți numerele:

94 și 52,

48 și -5,

-104 și -12

în cod complementar pe 8 biți.

3. Înmulțiți numerele:

9 și 11,

5 și 15,

-12 și 6

în cod complementar pe 8 biți.

La sfârșitul fiecărei operații spuneți dacă rezultatul este corect. Justificați-vă răspunsul.

Unitatea de învățare M2.U3. Reprezentarea numerelor raționale și a caracterelor

Cuprins

M2.U3.1. Introducere.....	41
M2.U3.2. Obiectivele unității de învățare.....	41
M2.U3.3. Reprezentarea numerelor reale	42
M2.U3.4. Reprezentarea caracterelor	45
M2.U3.5. Rezumat.....	46
M2.U3.6. Test de evaluare a cunoștințelor	47



M2.U3.1. Introducere

Calculatoarele lucrează cu numere reale approximate prin numere raționale. Acestea pot fi reprezentate în virgulă fixă sau în virgulă mobilă. În reprezentarea în virgulă fixă este nevoie un număr mare de biți pentru a putea memora un interval rezonabil de numere. De aceea, este preferată reprezentarea în virgulă mobilă. Majoritatea calculatoarelor folosesc pentru reprezentarea numerelor reale în virgulă mobilă standardul IEEE 754.

În finalul acestei unități de învățare, vom prezenta câteva coduri folosite pentru reprezentarea caracterelor: ASCII, EBCDIC, Unicode.



M2.U3.2. Obiectivele unității de învățare

Această unitate de învățare își propune ca obiectiv principal însușirea de către studenți a capacității de a determina reprezentarea numerelor reale (raționale) în virgulă mobilă în standardul IEEE 754. De asemenea, studenții se vor familiariza cu diferite coduri pentru reprezentarea caracterelor: ASCII, EBCDIC, Unicode.

La sfârșitul acestei unități de învățare studenții vor fi capabili să:

- Să determine reprezentarea în virgulă mobilă în standardul IEEE 754 în precizie simplă a unui număr rațional dat.
- Să determine reprezentarea în virgulă mobilă în standardul IEEE 754 în precizie dublă a unui număr rațional dat.
- Să determine numărul rațional, știind reprezentarea sa în virgulă mobilă în standardul IEEE 754 în precizie simplă sau dublă.



Durata medie de parcurgere a celei de-a treia unități de învățare este de 3 ore.

M2.U3.3. Reprezentarea numerelor reale

Calculatoarele lucrează cu numere reale approximate prin numere raționale. Există două abordări în ce privește reprezentarea numerelor raționale:

- Reprezentarea în virgulă fixă
- Reprezentarea în virgulă mobilă

În *reprezentarea în virgulă fixă*, după cum sugerează și numele reprezentării, există o poziție fixă a virgulei, un număr fixat de biți la stânga virgulei și un număr fixat de biți la dreapta virgulei. Dezavantajul major al reprezentării în virgulă fixă constă în faptul că în această reprezentare este nevoie un număr mare de biți pentru a putea memora un interval rezonabil de numere.

Să considerăm numărul: $0.10111_{(2)} * 2^4_{(10)}$. *Mantisa* sa este 0.10111, iar *exponentul* este 4. Exponentul indică ordinul de mărime al numărului printr-o putere a bazei, iar mantisa exprimă mărimea numărului în cadrul ordinului respectiv.

Folosind reprezentarea în virgulă mobilă, se pot memora pe același număr de biți, mai multe numere decât în reprezentarea în virgulă fixă deoarece pentru numerele “mari” se alocă mai mulți biți pentru exponent și mai puțini pentru mantisă, iar la cele “mici” se alocă mai puțini biți pentru exponent și mai mulți pentru mantisă.

O problemă care ar putea apărea când se lucrează cu numere reprezentate în virgulă mobilă constă în faptul că există multiple reprezentări ale aceluiași număr, ceea ce face ca operațiile aritmetice să devină mai dificile.

De exemplu, să considerăm din nou numărul: $0.10111_{(2)} * 2^4_{(10)}$. Acesta poate fi scris și într-una din următoarele forme:

$$0.10111_{(2)} * 2^4_{(10)} = 1.0111_{(2)} * 2^3_{(10)} = 101.11_{(2)} * 2^1_{(10)} = 10111_{(2)} * 2^{-1}_{(10)} \text{ etc.}$$

Pentru a evita multiplele reprezentări ale aceluiași număr, se poate conveni, de exemplu, să se așeze virgula în fața celei mai semnificative cifre nenule, iar exponentul să se modifice corespunzător cu deplasarea virgulei. Pentru reprezentarea lui 0, se va face o excepție de la această regulă. Dacă folosim această reprezentare atunci, dintre toate formele echivalente ale numărului de mai sus, se va alege următoarea: $0.10111_{(2)} * 2^4_{(10)}$.

Calculatoarele lucrează cu numere reale (raționale) reprezentate, în general, în virgulă mobilă în *formă normalizată*.

În *forma normalizată*, virgula se așează după cea mai semnificativă cifră nenulă și exponentul se modifică corespunzător cu deplasarea virgulei. Folosind această modalitate de reprezentare, se câștigă un bit suplimentar pentru mantisă. Pentru reprezentarea lui 0, se va face o excepție de la această regulă. În general, există câteva valori ale exponentului care sunt rezervate pentru cazuri speciale: +0, -0, +∞, -∞, NaN (Not a Number). Dacă folosim forma normalizată atunci, dintre toate formele echivalente ale numărului de mai sus, cea utilizată va fi $1.0111_{(2)} * 2^3_{(10)}$.



Exemple

$1.101_{(2)} * 2^7_{(10)} = 1.101_{(2)} * 10^{11}_{(2)}$ este un număr binar normalizat.

Majoritatea calculatoarelor folosesc pentru reprezentarea numerelor reale în virgulă mobilă standardul IEEE 754.

Standardul IEEE 754 în precizie simplă pentru reprezentarea numerelor reale în virgulă mobilă are următoarea structură:

- Cel mai semnificativ bit este bitul de semn, care dacă este 0 indică un număr pozitiv, iar dacă este 1 indică un număr negativ.
- Următorii 8 biți conțin exponentul, care este memorat adunat cu 127. Pentru acești 8 biți, valorile 00000000 și 11111111 sunt rezervate pentru cazurile speciale.
- Ultimii 23 de biți sunt destinați memorării părții fracționare, care, împreună cu bitul 1 din stânga virgulei, formează un număr cu 24 de biți: $1.\underbrace{ff \dots fff}_{23}$, unde

$\underbrace{ff \dots fff}_{23}$ reprezintă partea fracționară pe 23 de biți.

Există 5 tipuri de numere care pot fi reprezentate:

1. *Numerele normalizate nenule*, care sunt în formatul descris mai sus.
2. *Zero*, care este reprezentat prin exponentul 00000000 și cu toți biții 0 în partea fracționară. Bitul de semn poate fi 0 sau 1. Deci, sunt două reprezentări pentru zero: +0 și -0.
3. *Infinitul* este reprezentat prin exponentul 11111111 și cu toți biții 0 în partea fracționară. Bitul de semn poate fi 0 sau 1. Deci, atât $+\infty$ cât și $-\infty$ pot fi reprezentați.
4. *NaN (Not a Number)*. Dacă încercăm să împărțim 0 la 0 sau ∞ la ∞ , atunci rezultatul nu este definit. Acest rezultat nedefinit va fi NaN, care este reprezentat prin exponentul 11111111, iar partea fracționară conținând orice șir de biți exceptând șirul de 23 de zerouri. Bitul de semn poate fi 0 sau 1.
5. *Numere nenormalizate*. În reprezentarea normalizată, există un interval destul de mare între 0 și cel mai mic număr strict pozitiv care poate fi reprezentat. Numere din acest interval pot fi reprezentate, dacă nu sunt în forma normalizată. Pentru reprezentarea numerelor nenormalizate, exponentul este întotdeauna -127, care, adunat cu 127, este reprezentat prin 00000000. Bitul de semn poate fi 0 (pentru numere pozitive) sau 1 (pentru numere negative). Cei 23 de biți ai părții fracționare conțin chiar numărul de reprezentat scris în baza 2. Deci, lipsește acel bit 1 din stânga virgulei așa cum aveau numerele nenule normalizate.



Exemple

Semn	Exponent	Parte fracționară	Valoare
0	10000101	10100...0	$+1.101_{(2)} * 2^6$
1	00000001	01100...0	$-1.011_{(2)} * 2^{-126}$ (-126 este cel mai mic)

			exponent pentru numere normalizate)
0	11111110	000...0	$+1.0_{(2)} * 2^{127}$ (127 este cel mai mare exponent pentru numere normalizate)
0	00000000	000...0	+0
1	00000000	000...0	-0
0	11111111	000...0	$+\infty$
1	11111111	000...0	$-\infty$
0	11111111	10100...	NaN
0	00000000	10...0	$+0.1_{(2)} * 2^{-127}$ (număr nenormalizat)
1	00000000	010...0	$-0.01_{(2)} * 2^{-127}$ (număr nenormalizat)



Scrieți reprezentarea în standardul IEEE 754 în precizie simplă a următoarelor numere: $+1.11101_{(2)} * 2^{13}$, $-1.01_{(2)} * 2^{-3}$, $+0.001_{(2)} * 2^{-127}$, $-0.101_{(2)} * 2^{-127}$.

Standardul IEEE 754 în precizie dublă pentru reprezentarea numerelor reale în virgulă mobilă are următoarea structură:

- Cel mai semnificativ bit este bitul de semn, care dacă este 0 indică un număr pozitiv, iar dacă este 1 indică un număr negativ.
- Următorii 11 biți conțin exponentul, care este memorat adunat cu 1023. Pentru acești 11 biți, valorile 00000000000 și 11111111111 sunt rezervate pentru cazurile speciale.
- Ultimii 52 de biți sunt destinați memorării părții fracționare, care, împreună cu bitul 1 din stânga virgulei, formează un număr cu 53 de biți: $1.\underbrace{ff \dots fff}_{52}$, unde

$\underbrace{ff \dots fff}_{52}$ reprezintă partea fracționară pe 52 de biți.

În standardul IEEE 754 în precizie dublă sunt aceleași 5 tipuri de numere care pot fi reprezentate ca și în standardul IEEE 754 în precizie simplă.

În standardul IEEE 754, în afară de formatul în precizie simplă și formatul în precizie dublă, există și formatul în precizie simplă extinsă și în precizie dublă extinsă. Aceste formate în precizie extinsă nu sunt vizibile utilizatorului, dar sunt folosite pentru a reține un număr mai mare de biți pe parcursul calculelor, astfel încât să se reducă efectul erorilor cauzate de rotunjirile care se efectuează în urma calculelor. În formatele în precizie extinsă se folosesc biți suplimentari atât pentru exponent cât și pentru partea fracționară. Numărul acestor biți suplimentari poate varia de la o implementare la alta. De exemplu, un format pentru reprezentarea numerelor reale în precizie simplă extinsă poate avea 3 biți în plus la exponent și 8 biți suplimentari la partea fracționară față de formatul în precizie simplă. În general, formatul pentru reprezentarea numerelor reale în precizie dublă extinsă are 80 de biți, din care 15 biți pentru exponent și 64 de biți pentru partea fracționară.

Observație Orice implementare a standardului IEEE 754 trebuie să conțină cel puțin reprezentarea în precizie simplă. Celelalte formate sunt opționale.

Pentru ca rezultatul operațiilor cu numere reale să fie cât mai exact, se folosesc biți suplimentari pentru partea fracționară pe parcursul calculelor și diferite metode de rotunjire a rezultatelor. În standardul IEEE 754 există 4 moduri de rotunjire:

1. Rotunjirea spre 0
2. Rotunjirea spre $+\infty$
3. Rotunjirea spre $-\infty$
4. Rotunjirea spre cel mai apropiat număr care poate fi reprezentat în formatul dat.

Rotunjirea implicită este cea spre cel mai apropiat număr care poate fi reprezentat în formatul dat.

M2.U3.4. Reprezentarea caracterelor

ASCII

ASCII este un acronim pentru American Standard Code for Information Interchange. ASCII a fost creat de către American National Standard Institute (ANSI).

ASCII este un cod binar folosit pentru codificarea caracterelor pentru ca acestea să poată fi transmise procesorului de către periferice (de exemplu, de la tastatură).

ASCII a fost conceput inițial ca un cod pe 7 biți, numit și ASCII-7, cu ajutorul căruia se puteau reprezenta 128 de caractere. Deoarece acum majoritatea calculatoarelor folosesc un byte (8 biți) pentru a reprezenta un caracter, codul ASCII a fost extins la 8 biți, prin adăugarea unui bit suplimentar la stânga. Acest bit suplimentar este adesea 0. Unele procesoare îl folosesc ca bit de control, iar altele îl folosesc pentru a dubla numărul de caractere care pot fi reprezentate.

ASCII are următoarele caracteristici:

1. Atât codurile cifrelor cât și ale literelor sunt consecutive. Adică, codul caracterului '1' este egal cu codul caracterului '0' + 1. De asemenea, codul lui 'B' este codul lui 'A' + 1 ș.a.m.d.
2. La fiecare 32 de caractere, începe o nou șir. De exemplu, șirul cu codurile literelor mici este imediat după șirul cu codurile literelor mari: 'A' + 32 = 'a', 'B' + 32 = 'b' etc.

EBCDIC

EBCDIC este un acronim pentru Extended Binary Coded Decimal Interchange Code și a fost creat de IBM.

EBCDIC este codul folosit pentru a trimite informații între dispozitivele periferice și procesor.

Fiecare cod EBCDIC conține două părți, fiecare constând într-un *nibble* (4 biți). Cel mai semnificativ nibble definește clasa în care se află caracterul, iar celălalt nibble definește un anumit caracter din cadrul clasei.

Cu ajutorul codului EBCDIC, care este un cod pe 8 biți, se pot reprezenta 256 de caractere. De exemplu:

- Cei 4 biți care indică clasa în care se află codurile literelor de la 'A' la 'I' sunt 1100, iar ceilalți 4 biți care definesc caracterele propriu-zise în cadrul clasei variază de la 0001 la 1001.

- Nibble-ul care indică clasa în care se află codurile literelor de la 'J' la 'R' este 1101, iar cei 4 biți care definesc caracterele propriu-zise în cadrul clasei variază de la 0001 la 1001.
- Cei 4 biți care indică clasa în care se află codurile literelor de la 'S' la 'Z' sunt 1110, iar ceilalți 4 biți care definesc caracterele propriu-zise în cadrul clasei variază de la 0001 la 1001.
- Cei 4 biți care indică clasa în care se află codurile cifrelor de la '0' la '9' sunt 1111, iar ceilalți 4 biți conțin echivalentul binar (pe 4 poziții) al cifrei de reprezentat.

Unicode

Deoarece calculatoarele lucrează cu numere, caracterele sunt memorate prin intermediul unor numere. Unicode asociază un număr unic fiecărui caracter, independent de platformă, program sau limbaj. Standardul Unicode a fost adoptat de Apple, HP, IBM, Microsoft, Oracle, SAP, Sun, Sybase, Unisys etc. Multe sisteme de operare, toate browserele și, de asemenea, XML, Java, WML etc. utilizează Unicode.

Unicode este produs de Unicode Consortium. Acest cod asigură baza pentru procesarea, stocarea datelor de tip caracter.

Unicode asociază fiecărui caracter un cod, care este un număr hexazecimal.

Codurile din Basic Multilingual Plane (BMP) sunt formate din 4 cifre hexazecimale (de exemplu, U+0030 este codul lui '0', U+0031 este codul lui '1',..., U+0039 este codul lui '9', U+0041 este codul lui 'A', U+0042 este codul lui 'B',..., U+005A este codul lui 'Z', U+0061 este codul lui 'a', U+0062 este codul lui 'b',..., U+007A este codul lui 'z'). Ca și în codul ASCII, și în Unicode atât codurile cifrelor cât și ale literelor sunt consecutive.

Codurile care nu fac parte din BMP sunt formate din 5 sau 6 cifre hexazecimale.



M2.U3.5. Rezumat

Există două modalități de reprezentare a numerelor reale: în virgulă fixă sau în virgulă mobilă. În prima reprezentare, după cum sugerează și numele, există o poziție fixă a virgulei, un număr fixat de biți la stânga virgulei și un număr fixat de biți la dreapta virgulei. În reprezentarea în virgulă mobilă, deoarece poziția virgulei nu este prestabilită, se pot memora, pe același număr de biți, mai multe numere decât puteau fi memorate folosind reprezentarea în virgulă fixă. Majoritatea calculatoarelor lucrează cu numere raționale reprezentate în virgulă mobilă în standardul IEEE 754 și cu numere reale approximate prin numere raționale.

În finalul acestei unități de învățare, am prezentat cele mai cunoscute coduri folosite pentru reprezentarea caracterelor ASCII și Unicode și am făcut o succintă descriere a codului EBCDIC.



M2.U3.6. Test de evaluare a cunoștințelor

Scrieți reprezentarea în standardul IEEE 754 în precizie simplă a următoarelor numere: $+1.10101_{(2)} * 2^{13}$, $-1.001_{(2)} * 2^{-13}$, $+0.101_{(2)} * 2^{-127}$, $-0.111_{(2)} * 2^{-127}$.



Temă de control

1. Determinați codurile directe pe 8 biți ale următoarelor numere:
14, 52, +0, -0, -121.
2. Determinați codurile inverse pe 8 biți ale următoarelor numere:
14, 52, +0, -0, -121.
3. Determinați codurile complementare pe 8 biți ale următoarelor numere:
14, 52, +0, -0, -121.
4. Adunați numerele:
29 și 62,
55 și -65,
-101 și -56
în cod complementar pe 8 biți.
5. Scădeți numerele:
84 și 52,
88 și -5,
-94 și -12
în cod complementar pe 8 biți.
6. Înmulțiți numerele:
6 și 11,
7 și 15,
-12 și 7
în cod complementar pe 8 biți.
La sfârșitul fiecărei operații spuneți dacă rezultatul este corect. Justificați-vă răspunsul.
7. Scrieți reprezentarea în standardul IEEE 754 în precizie simplă a următoarelor numere: $+1.10101_{(2)} * 2^{33}$, $-1.0101_{(2)} * 2^{-3}$, $+0.1001_{(2)} * 2^{-127}$, $-0.0101_{(2)} * 2^{-127}$.